Technical Report: Fraudulent Claim Detection

Global Insure - Machine Learning Assignment

Executive summary

This report documents a full analytical and modeling workflow to support **early detection of fraudulent auto insurance claims** for Global Insure. Using a labeled dataset of **1,000 claims** and **approximately 40 attributes** per claim, the work follows a structured pipeline: data preparation and cleaning, stratified **70/30** train–validation split, exploratory analysis, **RandomOverSampler** balancing on the training set only, feature engineering and dummy encoding, and comparison of **Logistic Regression** (with **RFECV** feature selection, statistical diagnostics, and **tuned probability cutoff 0.57**) versus a **tuned Random Forest**.

On **held-out validation** data (natural class distribution, not oversampled), **Logistic Regression** achieves stronger **recall** and **F1-score** than Random Forest while maintaining high **specificity**, and is therefore recommended as the preferred model for fraud triage. Random Forest showed **near-perfect training fit** with **cross-validation** evidence of **overfitting**, underscoring the importance of validating on representative holdout data.

---

1. Introduction and problem statement

1.1 Context

Global Insure processes large volumes of claims annually. A non-trivial share of claims is **fraudulent**, leading to direct financial loss. The existing process relies heavily on **manual inspection**, which is slow, costly, and often identifies fraud **too late**-sometimes after substantial payouts have occurred.

1.2 Problem Statement

The organization seeks **data-driven** methods to **classify claims as fraudulent or legitimate earlier** in the approval workflow, in order to:

- Reduce paid fraud and investigation waste
- Prioritize specialist fraud (SIU) resources
- Improve consistency and scalability compared with manual-only review

1.3 Business objective

Build a supervised learning model that predicts **fraud reported** (`Y` / `N`) from historical **claim details**, **policy attributes**, **incident characteristics**, and **customer profile** fields, so that **new claims** can be scored **before** final approval.

1.4 Guiding analytical questions

The assignment frames four questions that the methodology and results address:

1. How can historical claim data be analyzed to reveal patterns associated with fraud?
2. Which features are most predictive of fraudulent behavior?
3. Can the **likelihood** of fraud for a new claim be estimated from past data?
4. What **operational insights** can models provide to improve the fraud detection process?

---

2. Data description

2.1 Scale and target

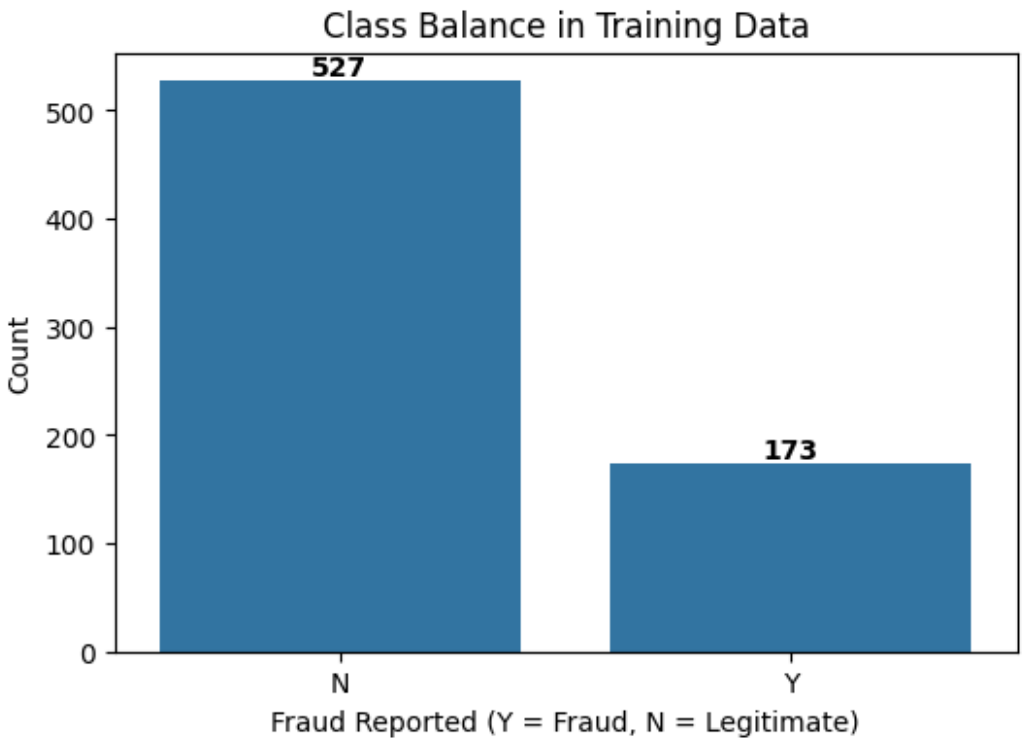
Aspect	Detail
Rows	1,000 claims
Columns	~40 (including target)
Target	`fraud_reported` - categorical (`Y` = fraud, `N` = legitimate)

The notebook documents a full **data dictionary** covering policy identifiers and dates, combined single limit (`policy_csl`), deductibles, premiums, umbrella limits, insured demographics (sex, education, occupation, hobbies, relationship), geographic fields, incident type/severity/collision, authorities contacted, witnesses, police report flag, claim amount components (`injury_claim`, `property_claim`, `vehicle_claim`, `total_claim_amount`), vehicle make/year, and the fraud label.

2.2 Class imbalance

Exploratory analysis shows **imbalanced** classes: **fraud (`Y`) is the minority**. After the stratified split, the training set exhibits approximately **24.7%** fraud prevalence-typical of fraud problems and requiring careful modeling and evaluation (see Section 6).

Figure 1. Distribution of the target variable on the training set (class imbalance).



---

3. Overall methodology

The assignment defines eight stages, executed in order:

1. **Data preparation** - load data, inspect structure and types.

2. **Data cleaning** - missing values, invalid encodings, derived fixes.

3. **Train–validation split (70/30)** - **stratified** on the target, fixed random seed for reproducibility.

4. **EDA on training data** - distributions, relationships to fraud, visual summaries.

5. **EDA on validation data** (optional) - consistency checks.

6. **Feature engineering** - time-based features, ratios, drops for redundancy.

7. **Model building** - Logistic Regression and Random Forest with documented sub-steps.

8. **Prediction and evaluation** - metrics on validation, comparison, and conclusion.

Principle: All **balancing** (oversampling) is applied **only to the training set**. The **validation set** retains the **original** claim mix so performance reflects **deployment-like** prevalence.

---

4. Data preparation and cleaning

4.1 Missing and sentinel values

- Missing counts were computed per column.
- The dataset uses **?** as a **sentinel for unknown/missing** in several categorical fields. These were converted to **NaN** for proper treatment.
- **Examples from the notebook:**
  - **`collision_type`:** many ? values → imputed using the **mode** (*Rear Collision*).
  - **`police_report_available`:** many ? values → imputed using the **mode** (*NO*).

4.2 Structural cleanup

- **Completely empty** columns (all NaN) were removed so they do not distort row-wise dropping or modeling.
- Rows with unresolved missingness were handled according to the notebook's drop/impute logic after sentinel replacement.
- **Target encoding:** `fraud_reported` was encoded for modeling (e.g. **1** for `Y`, **0** for `N`) where required by algorithms.

4.3 Anomaly checks (EDA-driven)

The notebook flags issues such as **negative `umbrella_limit`** values where relevant, supporting informed cleaning or treatment before modeling.

---

5. Train–validation split

Parameter	Value
Split ratio	**70% train / 30% validation**
Stratification	**Yes** - preserves fraud rate in both splits
Random state	**42** (reproducibility)
Approximate sizes	**700** training rows, **300** validation rows

Rationale: Stratification avoids a validation fold that is **unrepresentatively** easy or hard for the minority class, which would mislead metric interpretation.

---

6. Exploratory data analysis (key techniques and findings)

6.1 Techniques used

- **Target distribution** - bar charts / value counts to confirm imbalance.
- **Numeric variables** - distributions and **boxplots** by `fraud_reported` to compare fraud vs non-fraud.
- **Categorical variables** - frequency tables and **fraud likelihood** by category (proportion of `Y` within each level).
- **Correlation-style views** - heatmaps among numeric features to spot redundancy and multicollinearity risks.
- **Cardinality** - identification of high-cardinality, low-signal fields for later dropping.

Figure 2. Distributions of selected numerical features (training data).

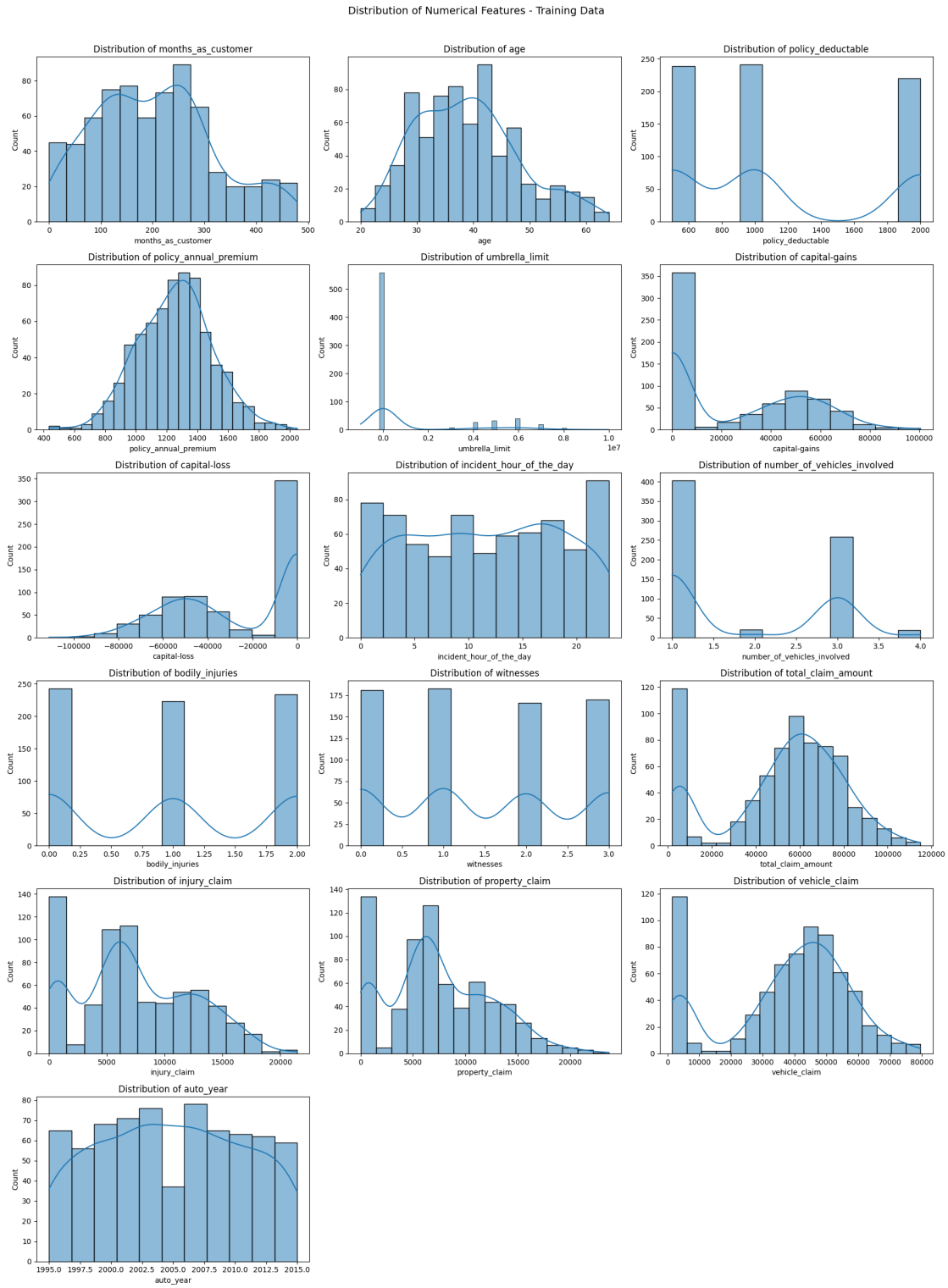


Figure 3. Correlation heatmap among numerical features (multicollinearity check).

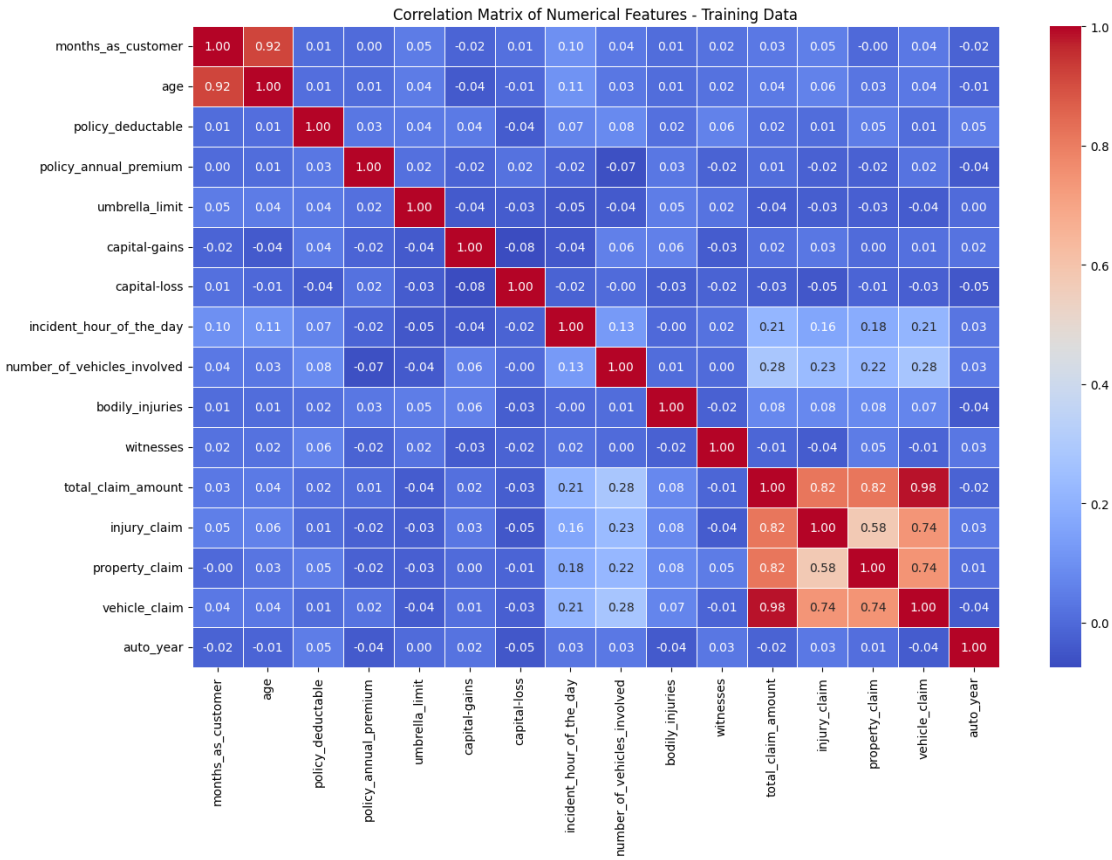


Figure 4. Example: fraud rate by category (target likelihood analysis for one categorical variable).

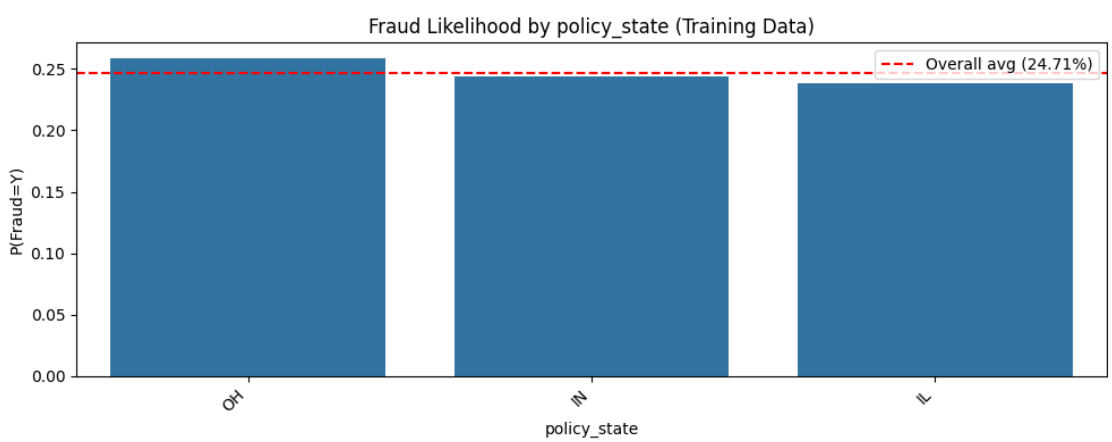
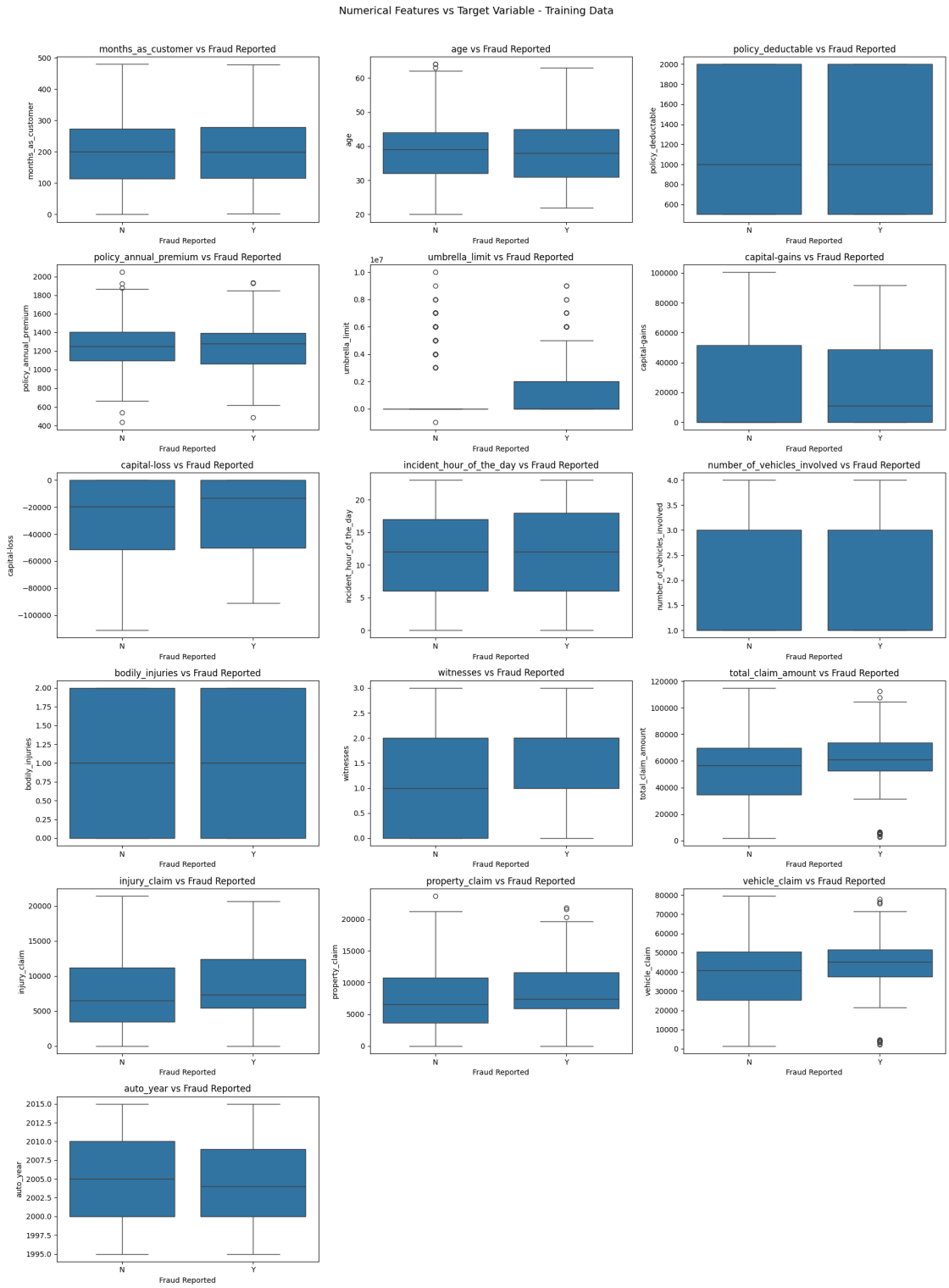


Figure 5. Selected numerical features vs. fraud label (boxplots).



6.2 Key insights

- **Imbalance** is material; a naive “always predict non-fraud” baseline would look deceptively accurate while failing the business goal.

- **Incident and documentation-related** categories (e.g. **incident type**, **collision type**, **police report availability**) show **different fraud rates** across levels-useful for both **investigator playbooks** and **model inputs**.

- **Claim composition** (injury vs property vs vehicle amounts) differs in ways that motivate **ratio features** rather than relying on totals alone.

- Validation EDA (where performed) supports **consistency** of patterns with training data, reducing concern about a single-fold fluke.

---

7. Handling class imbalance (training only)

Method: RandomOverSampler from `imblearn` - random duplication of minority-class rows until classes are balanced **on the training set**.

Effect: Training row count increased from **700** to **1,054** after resampling (and subsequent feature steps in the notebook use this expanded training matrix for fitting).

Critical safeguard: The **validation set (300 rows)** is **not** oversampled, so **precision, recall, F1, and accuracy** on validation reflect **real-world-like** class proportions.

---

8. Feature engineering

8.1 Time and tenure features

From raw dates (e.g. policy bind, incident):

- **`policy_age_days`** - tenure of the policy at relevant time.

- **`incident_month`**, **`incident_dow`** - calendar structure (seasonality / day-of-week effects).

- **`policy_bind_year`** - cohort effects.

8.2 Claim mix ratios

Because `total_claim_amount` equals the sum of component claims, using **both** total and components creates **strong linear dependence**. The notebook:

- Computes **`injury_ratio`**, **`property_ratio`**, **`vehicle_ratio`** as each component divided by **`total_claim_amount`** (before later dropping the total-see below).

- These ratios describe **how the claim is allocated** across injury, property, and vehicle buckets-often more discriminative than scale alone.

8.3 Drops for redundancy and noise

The notebook **drops** (from both train and validation, after alignment):

- **`policy_bind_date`**, **`incident_date`** - raw dates after features are extracted.
- **`total_claim_amount`** - redundant with components and harmful to multicollinearity.
- **`incident_city`** - **high cardinality** with **limited predictive value** in EDA.

After engineering and drops, the training feature matrix before encoding was on the order of **37 columns** (as printed in the notebook).

---

9. Encoding categorical variables

- **One-hot (dummy) encoding** via `pandas.get_dummies` on the agreed list of categorical columns, with **`drop_first=True`** to reduce **dummy variable trap** / linear dependence among dummies.
- Training and validation were aligned so the same dummy structure applies at scoring time (with handling for unseen categories as implemented in the notebook).

Post-encoding size (training): 102 numeric columns (including dummies).

---

10. Model 1: Logistic regression

10.1 Motivation

- **Interpretable** coefficients (after scaling context).
- Natural **probability outputs** for threshold tuning and risk ranking.
- Fits well with **statistical checks** (p-values, **VIF**).

10.2 Feature selection - RFECV

- **Algorithm: Recursive Feature Elimination with Cross-Validation (RFECV)**.
- **Base estimator:** `LogisticRegression` (**L-BFGS**, `max_iter=1000`, `random_state=42`).

- **CV: StratifiedKFold**, **5 folds**, `shuffle=True`, `random_state=42`.
- **Scoring: ROC-AUC** (appropriate for ranking quality under imbalance).
- **Result: 23** features selected as optimal (from **102** encoded columns).

10.3 Statistical model - Statsmodels

- Selected columns form **`X_train_log`**; a **constant** is added for intercept.
- **Statsmodels `Logit`** fits the logistic model on **encoded training labels**.
- Outputs include **coefficients**, **standard errors**, **z-stats**, **p-values**, and overall fit summaries (e.g. **pseudo R-squared** reported ≈ **0.541** in notebook output).

10.4 Multicollinearity - VIF

- **Variance Inflation Factor (VIF)** computed for predictors in the logistic specification.
- **Purpose:** detect **high linear dependence** among regressors; high VIF suggests unstable coefficients and redundant predictors.
- The notebook documents sorted VIFs and proceeds when values are in an acceptable range (with optional iterative dropping noted in the assignment).

10.5 Probability cutoff tuning

- Default **0.5** cutoff is **not** assumed optimal for fraud.
- The notebook tunes the decision threshold using training diagnostics (e.g. ROC / precision–recall considerations).
- **Chosen validation cutoff: 0.57** (reported in the conclusion).
- Training accuracy at **0.5** was reported around **0.878**; the **0.57** cutoff is selected to better trade off **false negatives** vs **false positives** for the fraud use case.

Figure 6. ROC curve for logistic regression (training data): trade-off between true and false positive rates.

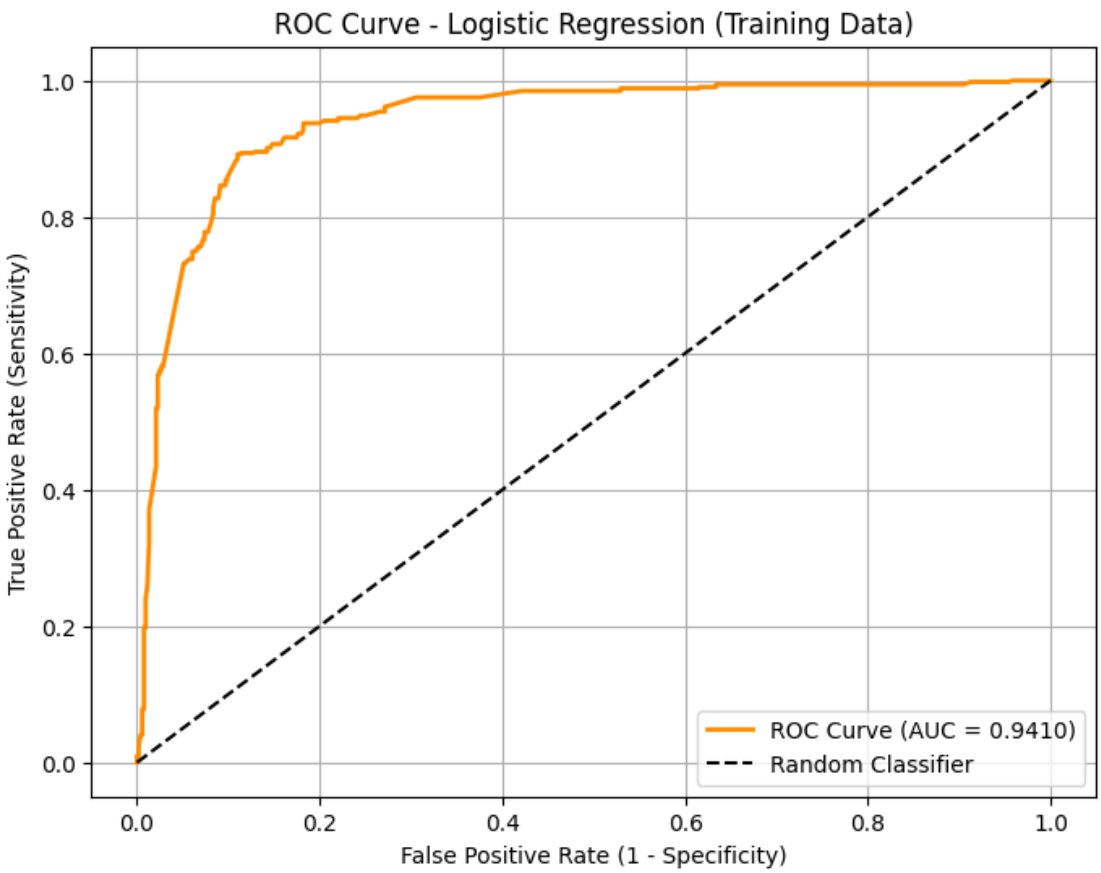


Figure 7. Accuracy, sensitivity, and specificity across probability cutoffs (logistic regression).

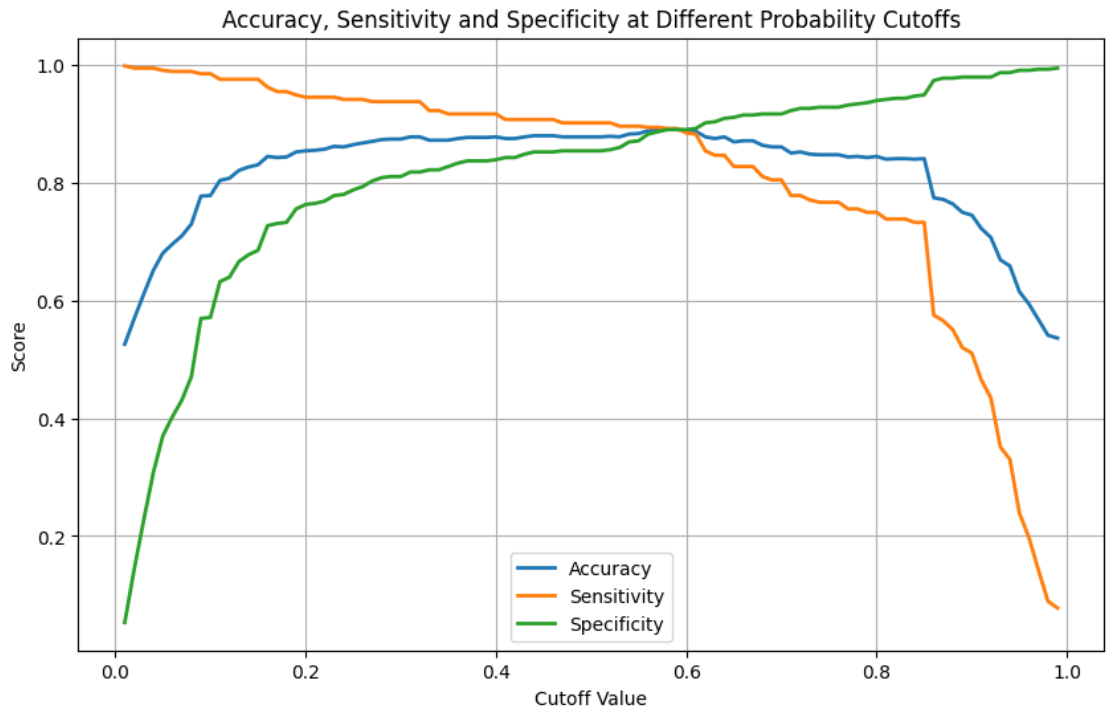
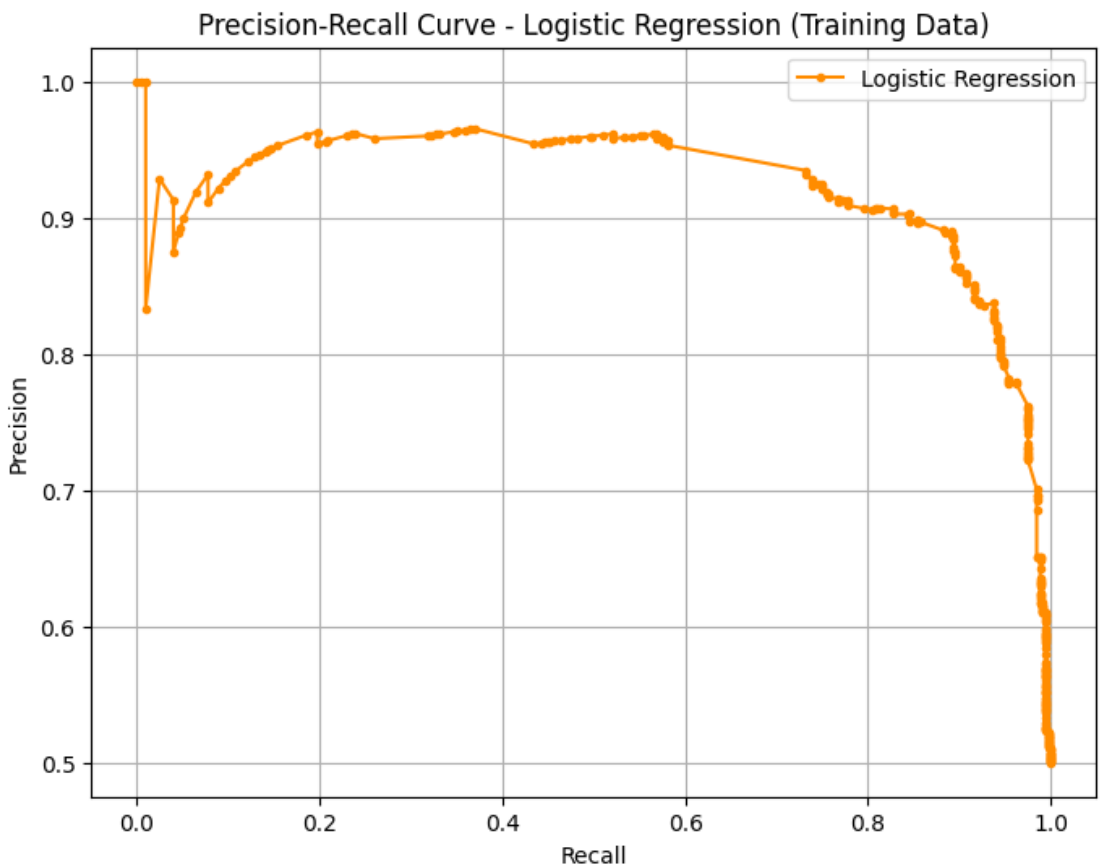


Figure 8. Precision–recall curve for logistic regression (training data).



11. Model 2: Random forest

11.1 Base model

- **RandomForestClassifier**, **100 trees**, `random_state=42`, parallel `n_jobs=-1`.

- Trained on full **102** dummy-encoded training features and **1,054** balanced rows.

11.2 Feature importance and subset selection

- **Feature importances** from the base forest were ranked.

- **Top 20** features (by importance) in the notebook include, among others:
  - **`incident_severity_Minor Damage`**, **`incident_severity_Total Loss`** (dummy levels)

  - **`insured_hobbies_chess`**

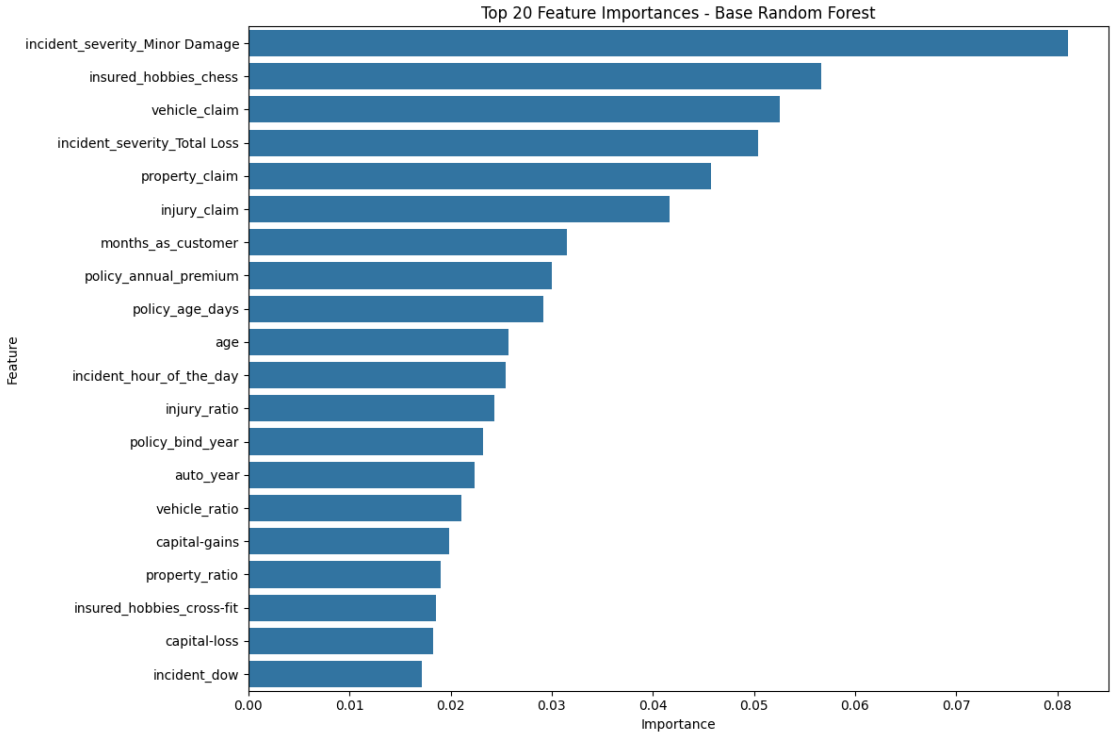
  - **`vehicle_claim`**, **`property_claim`**, **`injury_claim`**

  - **`months_as_customer`**, **`policy_annual_premium`**, **`policy_age_days`**, **`age`**

  - **`incident_hour_of_the_day`**, **`injury_ratio`**, **`policy_bind_year`**, **`auto_year`**

  - **`vehicle_ratio`**, **`capital-gains`**, **`property_ratio`**, **`insured_hobbies_cross-fit`**, **`capital-loss`**, **`incident_dow`**
- **Subset for tuned model:** all features with importance **≥ 0.01** → **22** features; training matrix **`(1054, 22)`**.

Figure 9. Random Forest: top feature importances (used to select predictors for the tuned model).



11.3 Cross-validation on training

- **5-fold cross-validation** (e.g. accuracy) on the selected-feature training set was used to assess **stability** vs **in-sample** performance.

- **Observation:** very high training performance with **CV metrics materially lower**, indicating **overfitting** risk for the forest on this sample.

11.4 Hyperparameter tuning - GridSearchCV

- **Base estimator:** `RandomForestClassifier(random_state=42, n_jobs=-1)`.

- **Parameter grid (illustrative from notebook):**
  - `n_estimators`: **[100, 200]**
  - `max_depth`: **[5, 10, None]**
  - `min_samples_split`: **[2, 5]**
  - `min_samples_leaf`: **[1, 2]**

- **CV folds: 5**

- **Scoring: ROC-AUC**

- **Best parameters found:** `n_estimators=100`, `max_depth=None`, `min_samples_split=2`, `min_samples_leaf=1`

A final forest **`rf_final`** was fit with these parameters on the **22**-feature training data.

---

12. Evaluation framework

12.1 Metrics reported (validation)

For **both** models, evaluation on **validation** used **binary predictions** derived from tuned thresholds (Logistic: **0.57**; Random Forest: default class prediction from the tuned forest as implemented in the notebook) and included:

- **Accuracy**

- **Sensitivity (Recall)** - fraction of actual frauds flagged

- **Specificity** - fraction of legitimate claims correctly cleared

- **Precision** - positive predictive value among predicted frauds

- **F1-score** - harmonic mean of precision and recall

Confusion matrices (TP, TN, FP, FN) were computed to support interpretation.

12.2 Why recall and F1 matter for fraud

Missing fraud (**false negatives**) typically costs **premium leakage** and lost subrogation; excessive false positives cost **customer friction** and **investigation load**. A good fraud model therefore emphasizes **recall** and **F1** while keeping **specificity** within acceptable bounds-exactly the logic stated in the notebook's conclusion.

---

13. Results (validation set)

Metric	Logistic Regression (cutoff **0.57**)	Random Forest (tuned)
Accuracy	**0.8533**	0.8067
Recall (sensitivity)	**0.7703**	0.5405

Metric	Logistic Regression (cutoff **0.57**)	Random Forest (tuned)
Specificity	0.8894	**0.8938**
Precision	**0.6951**	0.6250
F1-score	**0.7308**	0.5797

Figure 10. Confusion matrix on the held-out validation set — logistic regression (cutoff 0.57).

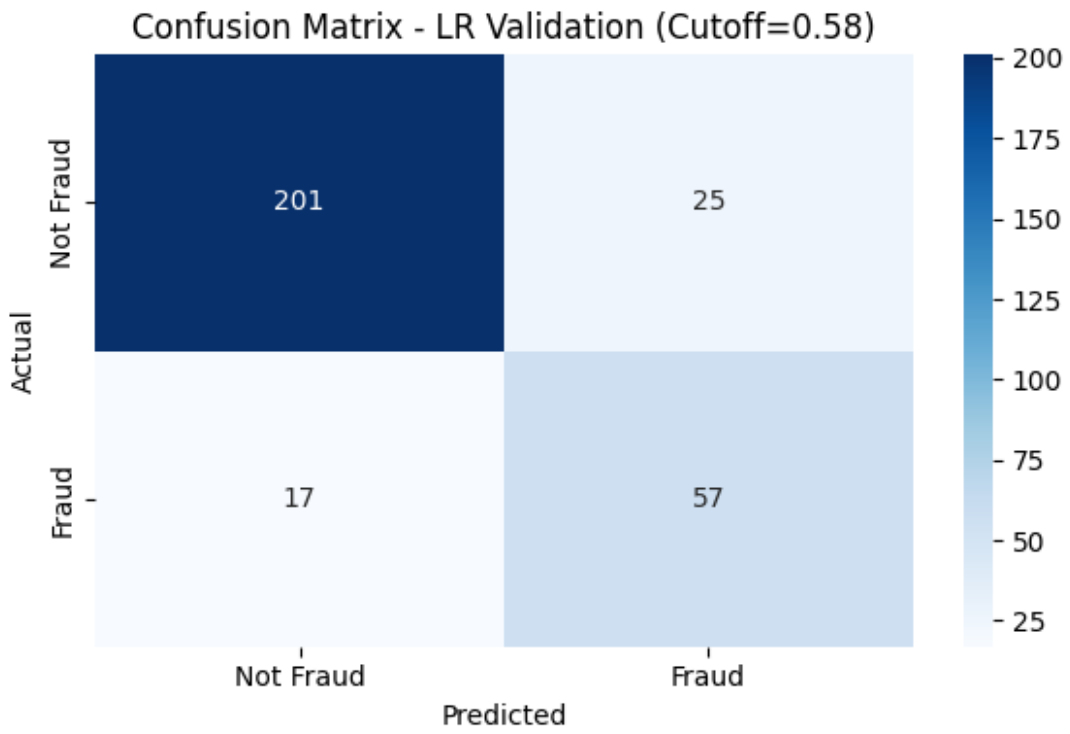
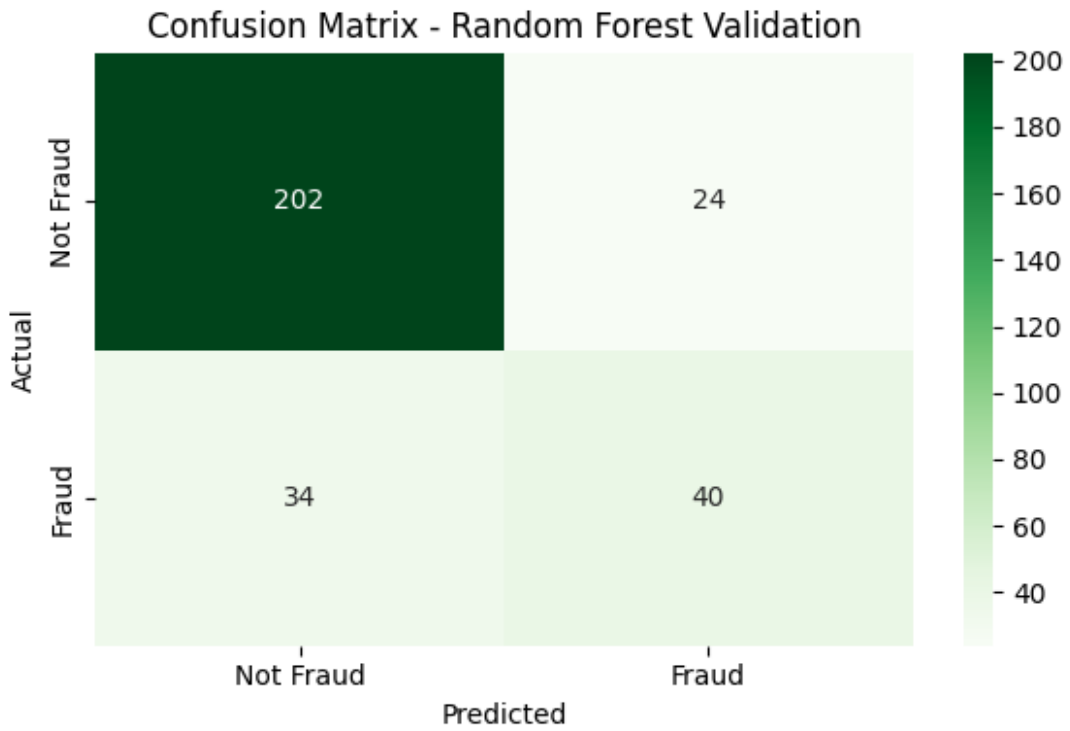


Figure 11. Confusion matrix on the held-out validation set — tuned random forest.



Interpretation:

- **Logistic Regression** identifies a **larger share of true frauds** (higher **recall**) and achieves a **clearly higher F1**, with **competitive specificity** (only slightly below the forest).

- **Random Forest** is slightly more conservative on negatives (**specificity**) but **misses more fraud** on validation.

- Random Forest's **near-perfect training performance** combined with weaker validation metrics supports a narrative of **overfitting** relative to the simpler, regularized linear model on this dataset size.

---

14. Conclusions and recommendations

14.1 Primary modeling conclusion

Adopt Logistic Regression with probability cutoff **0.57** as the **primary scoring model** for this assignment context: it **generalizes better** on holdout data for fraud-oriented metrics while remaining **interpretable** and **calibratable**.

14.2 Answers to the four guiding questions

1. **Patterns in historical data** - EDA combining **imbalance analysis**, **category-level fraud rates**, **numeric contrasts by fraud label**, and **correlation structure**, plus **domain-driven** feature construction (ratios, dates).

2. **Most predictive features** - **RFECV** selected **23** strong predictors for logistic regression; Random Forest **importance** highlights **severity dummies**, **hobby-related** dummies, **claim amounts and ratios**, **tenure/premium/age**, and **temporal** features.

3. **Predicting likelihood** - **Yes**; both models produce **scores** or **probabilities**; logistic outputs are especially suited to **ranked queues** and **threshold** tuning.

4. **Process insights** - **Tune thresholds** to organizational cost of FP vs FN; prefer models that **validate** well over those that **memorize** training; use **drivers** (coefficients / importances) to design **document requests**, **rules**, and **investigator training**.

14.3 Operational recommendations

- Deploy scores as **triage** (e.g. high / medium / low risk), not as sole grounds for denial, unless legally and operationally approved.

- **Monitor** precision/recall and **dollar-weighted** outcomes over time; **retrain** when fraud mix or products shift.

- Consider **time-based** validation (train on older claims, test on newer) in production-scale extensions to reduce **temporal leakage**.

- Document **fairness** review if protected attributes appear in or proxy through features.

---

15. Limitations

- **Sample size (n=1,000)** limits complexity of high-variance models (Random Forest).

- **Single random split**; **k-fold** or **repeated** validation would strengthen stability claims.

- **Random oversampling** duplicates minority rows-**SMOTE** or **class weights** are alternatives worth comparing.

- **External validity** to other regions, products, or channels is **not** established.